

Exercise 02
Unconstrained Control

Exercise 1 Finite vs Infinite Horizon LQR (Implementation)

Consider the discrete-time LTI system defined by

$$\begin{aligned}x(k+1) &= \begin{bmatrix} 4/3 & -2/3 \\ 1 & 0 \end{bmatrix} x(k) + \begin{bmatrix} 1 \\ 0 \end{bmatrix} u(k) \\ y(k) &= Cx(k), \quad C = \begin{bmatrix} -2/3 & 1 \end{bmatrix},\end{aligned}$$

and the cost

$$V = \sum_{i=0}^{N-1} (x_i^T Q x_i + u_i^T R u_i) + x_N^T P_N x_N,$$

with

$$Q = C^T C + 0.001 I_{2 \times 2}$$

$$R = 0.001$$

$$P_N = Q.$$

A:

- 1) By using the recursive approach, numerically compute the finite horizon controller with a horizon length of $N = 5$ and the resulting closed-loop state trajectory starting from the initial condition $x(0) = [10 \ 10]^T$.
- 2) Find the minimum horizon length $\bar{N}$ such that by applying the finite horizon controller in a receding horizon fashion the closed-loop system with initial condition $x(0)$ converges to the origin.
- 3) Plotting the prediction, motivate why increasing the horizon leads to convergence of the closed-loop system.
- 4) Given a horizon length $\bar{N}$ that leads to convergence of the closed-loop system, can you be sure that the system will also converge for $N > \bar{N}$?

B:

- 1) Implement the infinite horizon LQR controller $u = F_\infty x$.
- 2) Compute the infinite horizon cost for the system in closed-loop with $u = F_\infty x$ and compare it with the infinite horizon cost for the system in closed-loop with $u = F_0 x$, where F_0 is the finite horizon control law as computed in A1) with horizon length $\bar{N}$ obtained in A2) applied in a receding horizon fashion. Which one gives a lower cost?

Hints:

- Use the `dlqr` Matlab function to compute the LQR controller
- You can approximate the infinite horizon cost for the closed-loop system numerically using a long state and input trajectory:

$$V_\infty = \sum_{i=0}^{\infty} (x_i^T Q x_i + x_i^T F^T R F x_i) \sim \sum_{i=0}^{1000} (x_i^T Q x_i + x_i^T F^T R F x_i).$$

Exercise 2 Finite Horizon Optimal Control (Implementation)

Consider the discrete-time dynamic system with the following state space representation:

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \end{bmatrix} = \begin{bmatrix} 0.77 & -0.35 \\ 0.49 & 0.91 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} + \begin{bmatrix} 0.04 \\ 0.15 \end{bmatrix} u(k) \quad (1)$$

We want to design a linear quadratic optimal controller for this system with a finite horizon $N = 50$. We initially set the following cost matrices:

$$Q = \begin{bmatrix} 500 & 0 \\ 0 & 100 \end{bmatrix}, \quad R = 1, \quad P = \begin{bmatrix} 1500 & 0 \\ 0 & 100 \end{bmatrix},$$

and assume that the initial state is $x(0) = [1, -1]^T$;

- 1) Determine the optimal set of inputs

$$U = \begin{bmatrix} U \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix}$$

through the Batch Approach, i.e. by writing the dynamic equations as follows:

$$\begin{aligned} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ \vdots \\ x_N \end{bmatrix} &= \begin{bmatrix} I \\ A \\ \vdots \\ \vdots \\ A^N \end{bmatrix} x(0) + \begin{bmatrix} 0 & \dots & \dots & 0 \\ B & 0 & \dots & 0 \\ AB & B & \dots & 0 \\ \vdots & \ddots & \ddots & 0 \\ A^{N-1}B & \dots & AB & B \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ \vdots \\ u_{N-1} \end{bmatrix} \\ &= S^x x(0) + S^u U, \end{aligned}$$

and using the formula:

$$U^*(x(0)) = -(S^{uT} \bar{Q} S^u + \bar{R})^{-1} S^{uT} \bar{Q} S^x x(0),$$

and calculate the optimal cost $J_0^*(x(0))$:

$$J_0^*(x(0)) = x(0)^T (S^{xT} \bar{Q} S^x - S^{xT} \bar{Q} S^u (S^{uT} \bar{Q} S^u + \bar{R})^{-1} S^{uT} \bar{Q} S^x) x(0).$$

Hint: To efficiently concatenate the matrices, use the Matlab commands `kron`, `repmat`.

- 2) Verify the results of 1) by solving an optimization problem. In fact, the cost can be written as a function of U as follows:

$$\begin{aligned} J_0(x(0), U) &= (S^x x(0) + S^u U)^T \bar{Q} (S^x x(0) + S^u U) + U^T \bar{R} U \\ &= U^T H U + 2x(0)^T F U + x(0)^T S^{xT} \bar{Q} S^x x(0), \end{aligned}$$

where $H := S^{uT} \bar{Q} S^u + \bar{R}$ and $F := S^{xT} \bar{Q} S^u$, and then minimized by solving an unconstrained minimization problem. Check that the optimizer U^* and the optimal cost $J_0(x(0), U^*)$ correspond to the ones determined analytically in 1).

Hint: Quadratic optimization problems can be solved in Matlab using `quadprog`.

Note: Linear constraints on the inputs could have been specified when calling `quadprog`. Computing the optimal control trajectory in this way is a very basic version of model predictive control. More in the later lectures.

- 3) Design the optimal controller through the recursive approach and determine the optimal state-feedback matrices F_k . Start from the Riccati Difference Equations, assuming that $P_N = P$, and compute recursively the P_k :

$$P_k = A^T P_{k+1} A + Q - A^T P_{k+1} B (B^T P_{k+1} B + R)^{-1} B^T P_{k+1} A, \quad (2)$$

and then calculate F_k as a function of P_{k+1} :

$$F_k = -(B^T P_{k+1} B + R)^{-1} B^T P_{k+1} A.$$

Compare the optimal cost $J_0^*(x(0)) = x(0)^T P_0 x(0)$ with point 1) and check that they are equal.

- 4) We want to compare the robustness of the two approaches in simulation. Hence, let us add a process disturbance $Dw(k)$ to the right-hand side of Equation (1). Assume the matrix D to be:

$$D = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix},$$

while the process $w(k)$ is a Gaussian white noise, with mean $m = 0$ and variance $\sigma^2 = 10$.

Consider the simulation length to be equal to N time steps and that the inputs u_k to the system are defined as follows:

$$u_k = \begin{cases} U[k+1] & \text{for Batch Approach} \\ F_k x_k & \text{for Recursive Approach} \end{cases}$$

where $U[k]$ is the k -th component of the vector U .

Plot a graph of the state evolution over time. What is the difference in the dynamic evolution? What happens if you modify the variance of the disturbance? Motivate your answer.

Hint: Create a Matlab function that simulates your dynamic system, i.e. has output $x(k+1)$ and inputs $x(k)$ and $u(k)$, and calculate $u(k)$ based on the two different approaches. For the generation of the white noise, we suggest the Matlab commands `randn` or `awgn`. Initialize the random number generator with the command `rng(0)`.

Exercise 3 Infinite Horizon cost-to-go

Consider a discrete-time autonomous system $x(k+1) = Ax(k)$ with the system matrix

$$A = \begin{bmatrix} 0.995 & 0.099 \\ -0.099 & 0.985 \end{bmatrix}.$$

Recall the definition of a quadratic cost function. Assume that our system is at state $x = [1; 0]$. We now want to compute the infinite horizon cost-to-go for the system with state cost matrix

$$Q = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}.$$

Use Matlab to complete the following tasks:

- 1) Verify that the given system is asymptotically stable.
- 2) Simulate the system for $N = 100$ time steps and calculate the value of the cost function $J_0(x(0))$ with $x_0 = [1; 0]$.
- 3) Compute the infinite horizon cost-to-go using the discrete-time Lyapunov function. Compare this result to the result of the previous task. Verify that the cost obtained with the simulation above converges to the one using the discrete-time Lyapunov function by using larger N .